AI Research Assistant: Technical Design & Evaluation Report

Prepared by: Generative AI Technical Lead | **Date:** June 2026 | **Target:** Technical Review Board

1. Executive Summary

This report details the architectural principles, algorithmic pipelines, and quantitative evaluation metrics of the **Techity AI Research Assistant**, a production-ready, local-first Retrieval-Augmented Generation (RAG) system. Designed to ingest heterogeneous documents (PDFs, DOCX, TXT) and answer domain-specific questions, the platform implements double-indexing, reciprocal rank fusion (RRF), conversational memory, and continuous observability tracing. By bypassing traditional JWT sign-in walls, the platform boots directly into an active, single-user profile to optimize low-latency operations while executing real-time LLM-as-a-judge quality assessments.

2. High-Level System Architecture

The system uses a single-container architecture to eradicate Cross-Origin Resource Sharing (CORS) friction and context overhead. The application layers are decoupled as follows:

- **Presentation Layer (SPA):** Built on vanilla HTML5/ES6 JS and CSS grid styling. Connects to the backend via Server-Sent Events (SSE) for low-latency streaming and renders live Ragas evaluation score trend charts.
- **Application Backend:** A FastAPI server that routes incoming files, orchestrates indexing chunk engines, runs the agentic rewriter, and schedules downstream asynchronous evaluation jobs.
- **Data Persistence Layer:** Embeds a local metadata/FTS SQLite database alongside a vector database (ChromaDB) to manage text embeddings and search vectors concurrently.

3. Core Ingestion Pipeline

The ingestion engine processes uploaded documents through three discrete stages: parser extraction, recursive chunking, and dual-index placement.

For PDF files, text is extracted page-by-page using *pypdf* to maintain page-number coordinates, allowing the assistant to return exact page-number metadata for inline citations. DOCX structures are processed chronologically using *python-docx*, using a character-count threshold to simulate relative page limits, and raw TXT files are decoded directly.

3.1 Recursive Character Chunking

To preserve structural context, the system employs a custom recursive character splitter. Unlike basic character-limit boundaries that disrupt sentence flow and break key semantics, this algorithm evaluates splitting delimiters hierarchically (\n\n, \n, space, and empty string). Ingestion parameters constrain chunk sizes to a maximum of 1,200 characters with a sliding overlap of 150 characters, ensuring that paragraph cohesion is preserved across logical boundaries.

3.2 Double-Indexing Architecture

Once a document chunk is isolated, the ingestion pipeline duplicates the item into two parallel datastores:

1. **ChromaDB:** The chunk is embedded using the models/gemini-embedding-001 model. The resulting vector coordinates are indexed within an HNSW collection alongside metadata filters (user_id, document_id, filename, page_number).
2. **SQLite FTS5:** Simultaneously, the raw text is indexed in an optimized virtual full-text search table, ensuring domain-specific jargon and exact keywords can be retrieved instantly using native BM25 scoring.

4. Hybrid Retrieval & Reciprocal Rank Fusion (RRF)

To guarantee high recall and guard against semantic drift, retrieval utilizes Reciprocal Rank Fusion (RRF) to merge vector similarity results and keyword matching. Semantic vectors capture the high-level intent of queries, while SQLite FTS5 retrieves exact acronyms, names, and formulas.

For every chunk *c*, the system merges rank positions from the two search passes using the formula:

```
RRF_Score(c) = 1 / (60 + Rank_vector(c)) + 1 / (60 + Rank_keyword(c))
```

The constant coefficient 60 mitigates the influence of outlier ranks. The retrieval pipeline ranks all candidates and returns the top 6 merged chunks to the model context. This configuration ensures that relevant citations are selected even when spelling variations or specific jargon are present.

5. Agentic Multi-Turn Conversational Memory

RAG systems often fail on subsequent prompts when queries refer back to prior conversation history (e.g., asking 'Who wrote this paper?' followed by 'What was their methodology?').

To maintain true context, we implement an LLM-based query condenser: before retrieving documents, the backend queries the database for the last 6 messages (3 turns) associated with the active session. The model generates a consolidated, standalone search query containing all contextual prerequisites. If the LLM call fails or times out, the system defaults to the user's original raw text query to ensure service continuity.

6. Citation Mapping & Observability Tracing

To prevent hallucinations and build user trust, responses feature direct, clickable footnotes (e.g., [1], [2]). As tokens stream via Server-Sent Events, the system extracts the used index placeholders, matches them with retrieved source documents, and serves a structured citations payload mapping back to the filename, page number, and original context paragraph.

6.1 Real-Time Ragas Evaluation

We implement custom, light-weight math engines to evaluate responses locally in real-time, avoiding heavy external package dependencies:

- **Faithfulness:** Assesses grounding. The system extracts logical assertions from the generated answer and checks if they are directly supported by the context, computing: *(Supported Statements / Total Statements)*.
- **Answer Relevance:** Measures alignment with the query. The judge generates three prospective user queries based on the generated response and computes the mean cosine similarity between their vector embeddings and the original prompt.

7. Performance Benchmarks & Quantitative Evaluation

The system has been evaluated against test documents of varying length. Performance profiles under load are detailed below:

Doc Size	Ingest Latency	Hybrid RRF Time	TTFT	Avg Faithfulness	Avg Relevance
1 Page (Resume)	1.8s	45ms	480ms	0.95	0.92
15 Pages (Paper)	4.2s	72ms	520ms	0.90	0.88
50 Pages (Report)	11.5s	115ms	610ms	0.88	0.85

8. Key Findings & Recommendations

1. **Hybrid Retrieval Benefits:** Combining ChromaDB and SQLite FTS5 yields significantly better results for specific names, numbers, and libraries than using semantic vector searches alone.
2. **Latency Overhead:** Database writes and local embedding generations represent the primary ingestion bottleneck. Processing embedding tasks asynchronously or in batches can optimize throughput.
3. **Ragas Scores:** High faithfulness scores (averaging 0.91) validate the effectiveness of strict system prompting in keeping model output grounded in retrieved document chunks.